

These TypeScript types provide a solid, strongly-typed foundation for orchestrating your WebAssembly Stockfish engine worker. The separation of concerns between raw engine events and contextualized application-level events is well-thought-out. Here is an architectural review of your types, highlighting what works perfectly and offering a few editorial adjustments to make this protocol bulletproof.

What Works Brilliantly

- **Clear Intent Separation (EngineSearchPurpose):** Differentiating between 'ai' (the bot making a move), 'hint' (the user asking for advice), and 'eval' (continuous background evaluation of the current position) is excellent. It allows your UI or orchestration layer to route engine outputs to the correct components easily.
- **Contextual Intersection (EngineEvent):** Intersecting EngineSearchContext with the specific event types is a great pattern. It ensures that when your app receives an evaluation or a best move, it knows *exactly* which request initiated it, avoiding race conditions if the user clicks around quickly or changes positions before the worker finishes processing.

Recommended Refinements & Enhancements

To make this type definition production-grade and bulletproof against edge cases, consider a few subtle improvements:

1. Make EngineReadyEvent Safe for Intersection

In your EngineEvent union, you have:

```
TypeScript
export type EngineEvent =
  EngineReadyEvent |
  EngineSearchContext & {
```

If the engine fires a ready event *while* a search is active, or if you ever map over these events globally, it's safer if **every** member of the EngineEvent union shares a common literal discriminant field (like type).

Right now, EngineReadyEvent has type: 'ready', but the others only get their type from the nested union. While TypeScript can handle this, explicitly breaking out the types or ensuring they handle status events gracefully is cleaner.

2. Standardize Engine Scores (Centipawns vs. Mate)

For EngineHintEvent, you have evaluation: number | null and mateIn: number | null. In chess

engine protocols (like UCI), scores are often bound together because a position is *either* evaluated in centipawns or it's a forced mate.

Instead of two independent nullable fields, you can use a unified Score type. This prevents the impossible state where both are null or both have values.

3. Account for Engine Initialization/Errors

Workers can crash, fail to load the heavy WASM file, or hit memory limits. Adding an error state to your event union will save you a lot of debugging headaches down the road.



The Optimized Type Definition

Here is the refined version incorporating these professional design adjustments:

TypeScript

```
export type EngineSearchPurpose = 'ai' | 'hint' | 'eval'
```

```
export interface EngineSearchRequest
```

```
  fen: string
```

```
  depth: number
```

```
  moveTime?: number
```

```
  purpose: EngineSearchPurpose
```

```
  requestId: number
```

```
}
```

```
export interface EngineSearchContext
```

```
  requestId: number
```

```
  purpose: EngineSearchPurpose
```

```
}
```

```
// Unified Score structure to avoid impossible states (e.g., both evaluation and mateIn being null)
```

```
export type EngineScore =
```

```
  type 'cp' value: number // Centipawns (e.g., +150 for +1.5)
```

```
  type 'mate' value: number // Moves until mate (e.g., -3 for forced mate in 3)
```

```
export interface EngineReadyEvent
```

```
  type 'ready'
```

```
}
```

```
export interface EngineErrorEvent
```

```

type 'error'
error string
}

export interface EngineEvaluationEvent {
type 'evaluation'
depth number
score EngineScore
}

export interface EngineBestMoveEvent {
type 'bestmove'
move string // UCI format (e.g., "e2e4")
ponder? string // Optional but highly recommended if Stockfish suggests a ponder move
}

export interface EngineHintEvent {
type 'hint'
rank number // 1 for best, 2 for second best, etc. (MultiPV)
move string
score EngineScore
}

// Raw events parsed directly from the UCI string output
export type ParsedEngineEvent =
| EngineReadyEvent
| EngineErrorEvent
| EngineEvaluationEvent
| EngineBestMoveEvent
| EngineHintEvent

// Contextualized events dispatched to your UI or state manager
export type EngineEvent =
| EngineReadyEvent
| EngineErrorEvent
| (EngineSearchContext & EngineEvaluationEvent)
| (EngineSearchContext & EngineBestMoveEvent)
| (EngineSearchContext & EngineHintEvent)

```



Pro Tip for Implementation

When your Web Worker parses the raw UCI strings from Stockfish (like `info depth 12 score cp 35 nodes...` or `bestmove e2e4 ponder e7e5`), you can map them into these clean types

immediately.

For instance, your worker's internal router can keep track of the *current* activeContext (requestId and purpose). When a raw line finishes parsing into a ParsedEngineEvent, the worker wraps it with the activeContext before posting it back to the main thread.